

FreeModbus 业务代码开发指南

1. 适用范围

本文适用于以下两个工程：

- `example/GD_Firmware_Template_v1_modbus`
- `example/GD_Firmware_Template_v2_modbus`

两个工程的 Modbus 业务层写法相同，差异仅在板级串口引脚：

工程	FreeModbus 接口	引脚
V1	USART2 TTL	TX=PB10, RX=PB11
V2	USART2 TTL	TX=PD8, RX=PD9

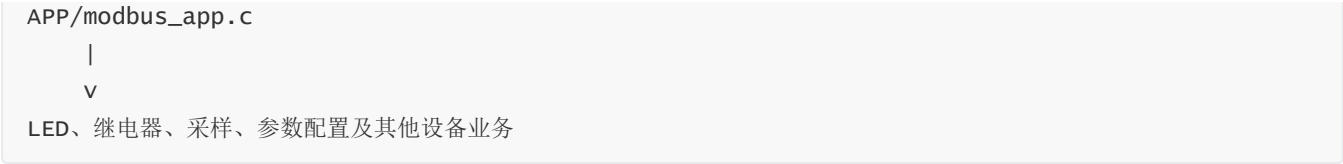
两者都使用以下通信参数：

- Modbus RTU 从站
- 从站地址 1
- 115200 bit/s
- 8 数据位、无校验、1 停止位
- USART2 直接 TTL 通信
- TIMER6 用于 RTU T3.5 帧间隔
- USART1 和 RS485 DE 控制保留给原有 `rs485_task`

2. 先理解业务代码的位置

当前工程的数据流如下：





各目录的职责：

目录或文件	职责	是否放业务
Components/modbus	FreeModbus 核心协议栈	否
Components/modbus/port	USART2、TIMER6、中断和事件适配	否
Components/modbus/app/mb_slave.c	从站初始化和轮询封装	否
Components/modbus/app/mb_registers.c	四类数据区的协议回调	仅做数据搬运和必要校验
Components/modbus/app/mb_registers.h	寄存器数量和寄存器地址定义	是，定义数据接口
APP/modbus_app.c	命令处理、状态更新、硬件控制	是，主要业务入口
APP/scheduler.c	周期调用业务任务	只添加任务入口
USER/src/main.c	初始化并持续调用 Modbus 轮询	不写具体业务

推荐新建 APP/modbus_app.c 和 APP/modbus_app.h。不要直接修改 FreeModbus 的 mb.c、mbrtu.c 或功能码实现文件来编写普通设备业务。

3. Modbus 中的“命令”是什么

本工程是 Modbus 从站。它不会主动向主站发送命令，而是等待电脑、PLC 或另一个 MCU 主站发起请求。

一般不需要创建自定义 Modbus 功能码。应当使用标准功能码读写寄存器，然后由业务任务解释寄存器的含义。例如：

- 主站写 40001=1：启动设备
- 主站写 40001=2：停止设备
- 主站写 40002=500：将目标值设置为 500
- 主站写线圈 00001：打开或关闭 LED1
- 主站读 30001：读取温度
- 主站读 30002：读取电压

当前已启用的标准功能码：

功能码	含义
01	读线圈

功能码	含义
02	读离散输入
03	读保持寄存器
04	读输入寄存器
05	写单个线圈
06	写单个保持寄存器
0F	写多个线圈
10	写多个保持寄存器
17	读写多个保持寄存器

4. 四类数据区如何选择

数据区	对外常用编号	权限	推荐用途
Coil	00001 起	读写, 1 bit	LED、继电器、启停开关
Discrete Input	10001 起	只读, 1 bit	按键、告警输入、限位状态
Input Register	30001 起	只读, 16 bit	温度、电压、计数值、状态
Holding Register	40001 起	读写, 16 bit	命令、参数、阈值、工作模式

当前每个工程提供：

```
#define MB_INPUT_REGISTER_COUNT      32U
#define MB_HOLDING_REGISTER_COUNT    32U
#define MB_COIL_COUNT                32U
#define MB_DISCRETE_INPUT_COUNT      32U
```

对应数组：

```
uint16_t mb_input_registers[32];
uint16_t mb_holding_registers[32];
uint8_t  mb_coils[4];
uint8_t  mb_discrete_inputs[4];
```

线圈和离散输入是按位打包的，所以 32 个 bit 只占用 4 个字节。

5. 地址 40001 和地址 0 的关系

Modbus 原始 PDU 地址从 0 开始，但很多上位机使用 40001、30001 等显示编号。

上位机显示	原始 PDU 地址	C 数组
00001	0	<code>mb_coils</code> 的 bit 0
10001	0	<code>mb_discrete_inputs</code> 的 bit 0
30001	0	<code>mb_input_registers[0]</code>
40001	0	<code>mb_holding_registers[0]</code>
40002	1	<code>mb_holding_registers[1]</code>

如果上位机要求填写“起始地址”，通常填写 0。如果上位机要求填写“寄存器编号”，通常填写 40001。必须根据上位机是否启用 PLC 地址模式进行确认。

FreeModbus 在调用项目回调前已经完成地址转换。当前 `mb_registers.c` 中的 `usAddress - 1U` 是正确处理，业务代码不应再额外加 1 或减 1。

6. 第一步：先设计寄存器表

不要一边写业务一边随意占用寄存器。先确定稳定的寄存器协议，并记录单位、符号、范围和读写权限。

以下是一份可以直接采用的示例：

6.1 输入寄存器

数组索引	对外编号	名称	类型	单位
0	30001	PT100 温度	int16	0.01 摄氏度
1	30002	ADC 电压	uint16	mV
2	30003	PT100 电阻	uint16	0.01 ohm
3	30004	设备状态	uint16	状态枚举
4	30005	故障码	uint16	故障枚举
5	30006	寄存器表版本	uint16	版本号

6.2 保持寄存器

数组索引	对外编号	名称	类型	说明
0	40001	命令码	uint16	0=无命令，1=启动，2=停止，3=复位
1	40002	命令参数	uint16	命令附带参数
2	40003	命令状态	uint16	0=空闲，1=执行中，2=完成，3=失败
3	40004	命令结果	uint16	0=成功，其他值为错误码

数组索引	对外编号	名称	类型	说明
4	40005	工作模式	uint16	设备业务定义
5	40006	温度阈值	int16	0.01 摄氏度

6.3 线圈和离散输入

数据点	名称	说明
00001	LED1 目标状态	主站可读写
00002	LED2 目标状态	主站可读写
10001	按键 1 状态	主站只读
10002	设备运行状态	主站只读
10003	故障状态	主站只读

建议把寄存器表作为设备通信协议的一部分进行版本管理。已经交付给上位机的地址不要随意改变含义。

7. 第二步：在 mb_registers.h 中定义索引

在两个工程各自的

Components/modbus/app/mb_registers.h 中增加业务索引。

对外文档使用 30001/40001，C 代码统一使用从 0 开始的数组索引。

```
/* Input Registers: read-only from the Modbus master's perspective. */
typedef enum {
    MB_IREG_TEMP_X100 = 0U,
    MB_IREG_VOLTAGE_MV,
    MB_IREG_RESISTANCE_X100,
    MB_IREG_DEVICE_STATUS,
    MB_IREG_FAULT_CODE,
    MB_IREG_MAP_VERSION
} mb_input_register_index_t;

/* Holding Registers: readable and writable by the Modbus master. */
typedef enum {
    MB_HREG_COMMAND = 0U,
    MB_HREG_COMMAND_PARAMETER,
    MB_HREG_COMMAND_STATUS,
    MB_HREG_COMMAND_RESULT,
    MB_HREG_WORK_MODE,
    MB_HREG_TEMP_LIMIT_X100
} mb_holding_register_index_t;

typedef enum {
    MB_COMMAND_NONE = 0U,
    MB_COMMAND_START = 1U,
```

```

    MB_COMMAND_STOP = 2U,
    MB_COMMAND_RESET = 3U
} mb_command_t;

typedef enum {
    MB_COMMAND_STATUS_IDLE = 0U,
    MB_COMMAND_STATUS_BUSY = 1U,
    MB_COMMAND_STATUS_DONE = 2U,
    MB_COMMAND_STATUS_ERROR = 3U
} mb_command_status_t;

#define MB_COIL_LED1      0U
#define MB_COIL_LED2      1U
#define MB_DISC_BUTTON1   0U
#define MB_DISC_RUNNING   1U
#define MB_DISC_FAULT      2U

#define MB_REGISTER_MAP_VERSION 0x0100U

```

使用枚举和宏以后，业务代码不再出现难以维护的

`mb_holding_registers[7]` 之类的“魔法数字”。

7.1 移除上电冒烟测试值

当前 `mb_registers_init()` 为了验证协议是否移植成功，会写入：

```

mb_input_registers[0] = 0x5678U;
mb_holding_registers[0] = 0x1234U;

```

开始使用本文的业务寄存器表时应删除这两行。否则

`mb_holding_registers[0]` 已经变成命令寄存器，上电后

`0x1234` 会被业务任务识别为未知命令。

保留四个 `memset()`，再由 `modbus_app_init()` 设置寄存器表

版本、命令状态和业务默认参数：

```

void mb_registers_init(void)
{
    memset(mb_input_registers, 0, sizeof(mb_input_registers));
    memset(mb_holding_registers, 0, sizeof(mb_holding_registers));
    memset(mb_coils, 0, sizeof(mb_coils));
    memset(mb_discrete_inputs, 0, sizeof(mb_discrete_inputs));
}

```

8. 第三步：创建 Modbus 业务任务

创建 `APP/modbus_app.h`：

```

#ifndef MODBUS_APP_H
#define MODBUS_APP_H

void modbus_app_init(void);
void modbus_app_task(void);

#endif

```

创建 `APP/modbus_app.c`。下面的例子演示：

- 把温度、电压、电阻更新到输入寄存器；
- 把线圈状态映射到 LED；
- 处理保持寄存器中的启动、停止和复位命令；
- 返回命令状态和结果。

```

#include "modbus_app.h"
#include "mb_registers.h"
#include "adc_app.h"
#include "led_app.h"

#include <stdint.h>

extern uint8_t ucLed[6];

static uint8_t prvBitGet(const uint8_t *pucBuffer, uint16_t usBit)
{
    return (uint8_t)((pucBuffer[usBit >> 3U] >>
                      (usBit & 7U)) & 0x01U);
}

static void prvInputRegistersUpdate(void)
{
    int16_t sTemperatureX100 =
        (int16_t)(PT100_temp * 100.0f);
    uint16_t usVoltageMV =
        (uint16_t)(adc_raw_voltage * 1000.0f);
    uint16_t usResistanceX100 =
        (uint16_t)(R_temp * 100.0f);

    mb_input_registers[MB_IREG_TEMP_X100] =
        (uint16_t)sTemperatureX100;
    mb_input_registers[MB_IREG_VOLTAGE_MV] =
        usVoltageMV;
    mb_input_registers[MB_IREG_RESISTANCE_X100] =
        usResistanceX100;
    mb_input_registers[MB_IREG_MAP_VERSION] =
        MB_REGISTER_MAP_VERSION;
}

static void prvOutputsUpdate(void)
{

```

```

    ucLed[0] = prvBitGet(mb_coils, MB_COIL_LED1);
    ucLed[1] = prvBitGet(mb_coils, MB_COIL_LED2);
}

static void prvCommandProcess(void)
{
    uint16_t usCommand =
        mb_holding_registers[MB_HREG_COMMAND];
    uint16_t usParameter =
        mb_holding_registers[MB_HREG_COMMAND_PARAMETER];

    if (usCommand == MB_COMMAND_NONE) {
        return;
    }

    mb_holding_registers[MB_HREG_COMMAND_STATUS] =
        MB_COMMAND_STATUS_BUSY;
    mb_holding_registers[MB_HREG_COMMAND_RESULT] = 0U;

    switch (usCommand) {
    case MB_COMMAND_START:
        /* Start the device here. */
        (void)usParameter;
        mb_holding_registers[MB_HREG_COMMAND_STATUS] =
            MB_COMMAND_STATUS_DONE;
        break;

    case MB_COMMAND_STOP:
        /* Stop the device here. */
        mb_holding_registers[MB_HREG_COMMAND_STATUS] =
            MB_COMMAND_STATUS_DONE;
        break;

    case MB_COMMAND_RESET:
        /* Reset business state here. Do not block for a long time. */
        mb_holding_registers[MB_HREG_COMMAND_STATUS] =
            MB_COMMAND_STATUS_DONE;
        break;

    default:
        mb_holding_registers[MB_HREG_COMMAND_RESULT] = 1U;
        mb_holding_registers[MB_HREG_COMMAND_STATUS] =
            MB_COMMAND_STATUS_ERROR;
        break;
    }

    /* Clearing the command allows the same command to be sent again. */
    mb_holding_registers[MB_HREG_COMMAND] =
        MB_COMMAND_NONE;
}

void modbus_app_init(void)
{

```



```

    mb_input_registers[MB_IREG_MAP_VERSION] =
        MB_REGISTER_MAP_VERSION;
    mb_holding_registers[MB_HREG_COMMAND_STATUS] =
        MB_COMMAND_STATUS_IDLE;
}

void modbus_app_task(void)
{
    prvInputRegistersUpdate();
    prvOutputsUpdate();
    prvCommandProcess();
}

```

示例中的启动、停止和复位注释需要替换成真实设备操作。不要在此处重复调用 `embPoll()`。

9. 第四步：加入初始化和调度

在 `USER/src/main.c` 中，Modbus 初始化成功后调用一次：

```

if (mb_slave_init() != MB_ENOERR) {
    my_printf(DEBUG_USART, "BOOT: Modbus RTU init failed\r\n");
} else {
    modbus_app_init();
}

```

需要包含：

```
#include "modbus_app.h"
```

然后在 `APP/scheduler.c` 中包含头文件，并把任务加入任务表：

```

#include "modbus_app.h"

static task_t scheduler_task[] =
{
    {led_task,      1,  0},
    {adc_task,     100, 0},
    {modbus_app_task, 10, 0},
    {oled_task,    10, 0},
    {btn_task,      5,  0},
    {uart_task,     5,  0},
    {rs485_task,   100, 0},
    {rtc_task,     500, 0},
};

```

10 ms 是业务任务周期，不是 Modbus 字节轮询周期。主循环中的 `mb_slave_poll()` 仍然必须尽可能频繁调用：

```
while (1) {  
    mb_slave_poll();  
    scheduler_run();  
}
```

10. 第五步：把文件加入 Keil

新建文件后，需要在 V1、V2 的 Keil 工程中分别操作：

1. 打开 `MDK/Project.uvprojx`。
2. 在 APP 文件组上选择 Add Existing Files to Group。
3. 添加 `APP/modbus_app.c`。
4. 确认 APP 已在 C/C++ Include Paths 中。
5. Rebuild Target。

如果忘记把 C 文件加入 Keil，常见链接错误是：

```
Undefined symbol modbus_app_init  
Undefined symbol modbus_app_task
```

11. 命令处理的正确方式

11.1 简单同步命令

执行时间很短的命令可以在一次 `modbus_app_task()` 中完成：

主站写命令和参数

- > 回调保存到保持寄存器
- > `modbus_app_task` 读取命令
- > 状态设为 `BUSY`
- > 执行业务
- > 状态设为 `DONE` 或 `ERROR`
- > 清除命令码

主站推荐按以下顺序操作：

1. 写入命令参数。
2. 最后写入命令码。
3. 轮询命令状态。
4. 状态为 `DONE` 或 `ERROR` 后读取命令结果。

也可以用功能码 10 在一个请求里同时写入命令码和参数。由于 `embPoll()` 和 `scheduler_run()` 在同一个主循环中顺序执行，业务任务不会在一个 Modbus 请求写到一半时读取寄存器。

11.2 长时间异步命令

电机运动、校准、升级或 Flash 擦除等操作不能在一个任务调用中阻塞完成。应使用状态机：

```
switch (soperationState) {
case OP_IDLE:
    if (new_command) {
        start_operation();
        soperationState = OP_RUNNING;
        command_status = MB_COMMAND_STATUS_BUSY;
    }
    break;

case OP_RUNNING:
    if (operation_failed()) {
        soperationState = OP_IDLE;
        command_status = MB_COMMAND_STATUS_ERROR;
    } else if (operation_finished()) {
        soperationState = OP_IDLE;
        command_status = MB_COMMAND_STATUS_DONE;
    }
    break;
}
```

每次任务只推进一步，然后立即返回，保证 Modbus 仍能及时响应。

12. 数据编码规则

12.1 推荐优先使用缩放整数

Modbus 一个寄存器是 16 bit。为了避免不同上位机的浮点字节序差异，推荐使用缩放整数：

实际值	寄存器值	规则
25.36 摄氏度	2536	温度乘 100
-10.25 摄氏度	-1025 对应 int16	有符号温度乘 100
3.300 V	3300	电压转换为 mV
100.25 ohm	10025	电阻乘 100

负温度写入方式：

```
int16_t sTemperatureX100 =
    (int16_t)(PT100_temp * 100.0f);
mb_input_registers[MB_IREG_TEMP_X100] =
    (uint16_t)sTemperatureX100;
```

上位机必须把该寄存器按有符号 16 位整数解析。

转换前应检查范围，避免超过 `-32768..32767` 或 `0..65535`。超范围时应钳位并设置故障码。

12.2 32 位整数

两个连续寄存器可以存放一个 32 位值。推荐明确规定高字在前：

```
uint32_t ulValue = 0x12345678UL;

mb_input_registers[0] =
    (uint16_t)(ulValue >> 16U);    /* 0x1234 */
mb_input_registers[1] =
    (uint16_t)(ulValue & 0xFFFFU); /* 0x5678 */
```

必须在协议文档中固定 word order。FreeModbus 已保证单个寄存器内部按高字节、低字节发送，但两个寄存器之间的先后顺序属于应用协议约定。

12.3 浮点数

只有在上位机和 MCU 对 IEEE-754 以及 word order 有明确共同约定时才直接传 `float`。否则继续使用缩放整数。

12.4 字符串

字符串可以每个寄存器放两个字节，但必须规定：

- 每个寄存器内字符顺序；
- 最大长度；
- 是否以 0 结尾；
- 未使用字节如何填充。

13. 线圈的位操作

`mb_coils` 和 `mb_discrete_inputs` 是位数组，不能使用

`mb_coils[index]` 直接访问第 `index` 个线圈。

推荐辅助函数：

```
static uint8_t mb_bit_get(const uint8_t *buffer, uint16_t bit)
{
    return (uint8_t)((buffer[bit >> 3U] >>
                     (bit & 7U)) & 0x01U);
}

static void mb_bit_set(uint8_t *buffer,
                      uint16_t bit,
                      uint8_t state)
{
    uint8_t mask = (uint8_t)(1U << (bit & 7U));
```

```

if (state != 0U) {
    buffer[bit >> 3U] |= mask;
} else {
    buffer[bit >> 3U] &= (uint8_t)~mask;
}
}

```

例如把按键状态提供给主站：

```

mb_bit_set(mb_discrete_inputs,
           MB_DISC_BUTTON1,
           button_pressed);

```

14. 多个模块控制同一硬件的问题

当前按键任务会修改 `ucLed`。如果 Modbus 业务任务也持续覆盖 `ucLed`，按键操作会立即被 Modbus 状态覆盖。

同一个输出必须规定唯一的数据所有者。可以选择：

1. Modbus 线圈作为 LED 目标状态，按键也修改相同的线圈；
2. 业务状态作为唯一来源，Modbus 和按键都提交命令；
3. 明确优先级，例如本地急停始终高于远程命令。

不要让两个周期任务各自无条件写同一个变量。

15. 回调中可以做什么

当前四个回调位于 `mb_registers.c`：

- `EMBRegInputCB()`
- `EMBRegHoldingCB()`
- `EMBRegCoilsCB()`
- `EMBRegDiscreteCB()`

回调是在主循环调用 `EMBPoll()` 时执行，不是在 USART2 中断中直接执行。

即便如此，回调仍应保持快速和确定。

适合放在回调中的内容：

- 地址范围检查；
- 高低字节转换；
- 从寄存器数组复制数据；
- 对写入值做很短的合法性检查；
- 设置待处理标志。

不适合放在回调中的内容：

- `delay_ms()`；
- 等待串口发送完成；

- 大块 Flash 擦写；
- 文件系统操作；
- 电机运动完成等待；
- 大量浮点运算；
- 再次调用 `embPoll()`；
- 修改 USART2 或 TIMER6 配置。

如果一个写入值非法，可以选择：

- 在回调中拒绝，并返回合适的 Modbus 错误；
- 先接收，然后由业务任务将命令状态设为 ERROR，并给出结果码。

普通设备命令推荐第二种，参数配置可以根据协议要求选择第一种。

16. 参数保存到 Flash

不要在每次 Modbus 写寄存器时立即写 Flash。主站可能连续发送多个写请求，这会阻塞通信并快速消耗 Flash 擦写寿命。

推荐流程：

```
Modbus 写配置寄存器
-> 业务任务校验
-> 更新 RAM 中的有效配置
-> 设置 config_dirty
-> 一段时间没有新修改
-> 后台任务统一保存到 Flash
```

保存期间也应采用非阻塞设计，或明确让主站看到 BUSY 状态。

17. 主站联调步骤

17.1 硬件连接

```
USB-TTL TX  -> 板端 USART2 RX
USB-TTL RX  <- 板端 USART2 TX
USB-TTL GND -> 板端 GND
```

USART2 是 TTL，不允许把 RS485 转换器的 A/B 线直接接入。

17.2 软件参数

使用 Modbus Poll、QModMaster 或 PLC：

```
Mode      = Modbus RTU
Slave ID  = 1
Baud      = 115200
Data bits = 8
Parity    = None
Stop bits = 1
```

17.3 先验证基础协议

未替换当前冒烟测试值前：

操作	请求帧	预期
读 40001	01 03 00 00 00 01 84 0A	返回 0x1234
读 30001	01 04 00 00 00 01 31 CA	返回 0x5678

采用本文的业务寄存器表后，可测试：

操作	功能码	原始地址	数据
读取温度 30001	04	0	1 个寄存器
写启动命令 40001	06	0	1
写停止命令 40001	06	0	2
打开 LED1 线圈 00001	05	0	FF00
读取命令状态 40003	03	2	1 个寄存器

写 40001=1 的完整 RTU 帧：

```
01 06 00 00 00 01 48 0A
```

写线圈 00001=ON 的完整 RTU 帧：

```
01 05 00 00 FF 00 8C 3A
```

18. 调试建议

推荐按层定位：

- 1. 确认 USART2 TX/RX 交叉连接并共地。
- 2. 确认主站参数为 115200 8N1、从站地址为 1。
- 3. 观察 `USART2_IRQHandler()` 是否进入。
- 4. 观察 `TIMER6_IRQHandler()` 是否在一帧结束后进入。
- 5. 观察 `mb_slave_poll()` 是否持续执行。
- 6. 在相应寄存器回调处断点，确认地址和功能码。
- 7. 观察 `mb_holding_registers` 或 `mb_coils` 是否变化。
- 8. 最后检查 `modbus_app_task()` 是否执行实际业务。

不要在 USART2 上使用 `printf` 输出调试文本，否则文本会混入 Modbus RTU 数据流。调试输出继续使用 USART0。USART1 仍由 `rs485_task` 使用。

19. 常见问题

主站能写寄存器，但硬件没有动作

寄存器回调只改变数组。检查是否已经创建并调度

`modbus_app_task()`，以及业务任务是否把数组值映射到硬件。

读取地址总是偏移一个

检查上位机使用的是 40001 编号还是 PDU 原始地址。原始地址 0 对应 40001，不要在业务代码中再次进行地址转换。

温度负数显示为很大的正数

上位机把寄存器按无符号数解释了。将该地址的数据类型设置为 `int16`，再除以 100。

写入命令只执行一次

业务任务执行后应把命令寄存器清零，或者增加命令序号。否则连续写入相同值时，边沿检测逻辑可能无法识别第二次命令。

Modbus 偶发超时

检查业务任务和其他调度任务是否存在长时间阻塞。主循环必须频繁返回

`mb_slave_poll()`。

Keil 编译通过但链接找不到业务函数

确认 `APP/modbus_app.c` 已加入 Keil 工程，而不只是存在于磁盘。

20. 是否需要自定义功能码

绝大多数设备业务都可以通过线圈和保持寄存器实现，不建议一开始就增加自定义功能码。标准功能码更容易被 PLC、Modbus Poll 和第三方软件支持。

只有在寄存器模型确实无法表达业务时，才考虑通过

`embRegisterCB()` 注册自定义功能码。此时必须另外定义：

- 请求和响应 PDU 格式；
- 长度和边界校验；
- Modbus 异常响应；
- 主站侧解析；
- 兼容性和版本策略。

自定义功能码属于协议扩展，不应替代正常的设备业务任务。

21. 提交前检查清单

- 已形成正式寄存器表，包含地址、类型、单位、范围、权限和默认值。
- 业务代码位于 APP 层，没有修改 FreeModbus 核心状态机。

- USART2 没有调试文本、DMA/IDLE 收包或 RS485 DE 控制。
- USART1 的 `rs485_task` 未被 Modbus 业务占用。
- TIMER6 未被其他功能重复使用。
- 回调和业务任务中没有长时间阻塞。
- 有符号数、32 位数和浮点数的编码规则已写入通信协议。
- 多个任务不会无条件修改同一个硬件状态。
- 新增 C 文件已加入 V1、V2 各自的 Keil 工程。
- 两个工程均完成 Rebuild，结果为 0 Error。
- 已使用真实主站完成读、写、非法地址和断线恢复测试。